MCQ + Short-Answer Solving Guide

Lecture-based patterns, elimination strategies, worked solutions and exam-style practice

Scope: Part 2 Lecture 1 (Processes), Lecture 2 (More on Processes), Lecture 3 (Redirection), Lecture 4 (Pipes), plus the Spring 2026 practice exam. This guide emphasizes the 30-point MCQ and 20-point short-FRQ portion. Full-length coding templates are intentionally separate.

Use it this way: First read the decision map. During practice, identify the question type, apply the 2-4 step method, then compare your answer with the worked result. All 32 multiple-choice examples are newly written, not copied exam questions.

SOURCE MAP

Source	Main content used
Lecture 1 - 52 slides	fork return values; process count; independent memory; PCB, scheduler; wait; zombie; process order
Lecture 2 - 44 slides	exec family and argv; failure handling; FILE* vs FD; open flags; lseek; inherited descriptors
Lecture 3 - 56 slides	stdin/stdout/stderr; shell redirections; dup/dup2; FD table; fork + exec redirection
Lecture 4 - 60 slides	pipe syntax; interprocess bytes; pipeline construction; close/EOF; PIPE_BUF; matrix-worker example
Spring 2026 practice exam - 8 exam pages	10 MCQs (30 pts), 4 short FRQs (20 pts), one longer coding problem (50 pts)

Important: The practice exam forbids using ChatGPT during the exam. It allows certain local/HuskyCT/Codespaces materials; confirm whether prewritten reference sheets are permitted before using this PDF in the exam room.

00 | First: how to solve an unfamiliar question

Step 1: Decide what is being tested. Choose the branch below before you calculate anything.

Clue in question	Use this method	Go to
fork(), PID, print order	Draw a separate process branch; track each variable and ordering constraint	Lecture 1 MCQs / FRQ 1
wait(), zombie, exit code	Track parent-child relationship and decode status only after WIFEXITED	L1 + FRQ 3
execl/p, execv/p, argv	Identify list vs vector and PATH vs explicit path; check successful exec returns?	L2 + FRQs 4-5
open(), read(), flags, lseek	Track file bytes, offsets, read return values, and null terminator	L2 + FRQs 6-8
dup(), dup2(), FD values	Draw 0,1,2,3,... descriptor table; update each call	L3 + FRQs 9-11
pipe(), EOF, pipeline	Draw arrows, assign read/write ends, close unused ends	L4 + FRQs 12-16

High-return MCQ elimination rules

Claim in an option	Usually correct interpretation
"Parent always goes first"	Reject: OS scheduling determines order without synchronization.
"Child sees fork return equal to its PID"	Reject: child sees 0, getpid() gives its real PID.
"Successful exec returns and prints next line"	Reject: successful exec never returns to original code.
"read(fd,buf,N) always returns N"	Reject: N is maximum requested; return may be less, 0, or -1.
"One pipe automatically broadcasts to multiple readers"	Reject: pipe bytes are consumed, not copied to each reader.
"Closing original fd undoes dup2"	Reject: duplicated target descriptor remains valid.
"Wait can wait on parent"	Reject: wait/waitpid reap children, not the caller's parent.

Fast short-FRQ response structure

Definition -> Mechanism -> Result. Example: "**waitpid()** waits for the specified child [definition]; the caller blocks until that child changes state under the given options [mechanism]; it returns a PID/status, enabling the parent to reap the child [result]."

> **For code tracing:** annotate each line with process / FD / offset / bytes. **For code writing:** write 4-10 essential lines and explain why the stopping condition and cleanup are correct.

01 | Lecture 1: Introduction to Processes - MCQ patterns

Look for **branching** from fork(), independent variables, scheduler-dependent order, and lifetime terms (orphan vs zombie).

Rule	Immediate test
fork return: child=0, parent>0, error=-1	Do not confuse child's fork return with getpid().
Independent memory after fork	Separate stack/heap values per process.
Unconditional k forks	Up to 2^k total processes if every path executes all k.
wait()/waitpid()	Child completes before waiting parent continues; otherwise output order may vary.

1. Fork return values

After a successful pid = fork(), which return value is observed in the new child?

A. A positive PID | B. 0 | C. -1 | D. The parent PID

Answer: B. The child sees 0; the parent sees the child's positive PID; -1 means failure.

Solving method: Look at which process the question names, then apply the three fork return cases.

Slide support: L1 6-16, 31-34

---

2. Sequential forks

Assuming all calls succeed, how many processes exist after fork(); fork(); when every process reaches both calls?

A. 2 | B. 3 | C. 4 | D. 8

Answer: C. The first fork changes 1 process into 2; the second is executed by both, giving 4.

Solving method: For an unconditional fork, double the number of currently running paths.

Slide support: L1 40-50

---

3. Conditional fork count

Original parent executes: if (fork() == 0) _exit(0); fork(); How many processes were created in total (excluding original)?

A. 1 | B. 2 | C. 3 | D. 4

Answer: B. The first child immediately exits. Only the original parent reaches the second fork; 2 children are created.

Solving method: Follow the branches; only count processes that reach each fork.

Slide support: L1 31-34; L2 27-32

---

4. Memory after fork

The parent sets x=5 before fork(); the child sets x=20 and exits. What is x in the parent if it never changes x?

A. 0 | B. 5 | C. 20 | D. Uninitialized

Answer: B. The parent and child have independent copies of ordinary process memory.

Solving method: Separate stack/heap variable values by process after a fork.

Slide support: L1 24-25, 35

---

5. Output ordering

A parent and child each print one line after fork(), with no waiting. Which statement is always true?

A. Parent first | B. Child first | C. Both print but order may vary | D. Neither prints

Answer: C. The OS scheduler determines which process runs first; assume the output calls execute and buffering is controlled.

Solving method: Search for wait(), a pipe dependency, or other synchronization before claiming an order.

Slide support: L1 33-34, 48-50

---

6. Zombie or orphan

Parent exits while its child is still running. What is the child called?

A. Zombie | B. Orphan | C. Terminated | D. A thread

Answer: B. A running child whose original parent has exited becomes an orphan (is reparented).

Solving method: Distinguish an actively running child from one that has already exited.

Slide support: L1 36-38

---

7. Wait and process state

A child has terminated and its parent has not reaped it. Which statement is correct?

A. Zombie | B. Orphan | C. Runnable | D. A new parent is required immediately

Answer: A. It has stopped executing but retains a process-table entry pending reaping.

Solving method: If dead + unreaped => zombie; if alive + parent gone => orphan.

Slide support: L1 36-38

---

8. PCB and OS scheduling

What most directly stores a process's saved execution state for OS management?

A. argv | B. A PCB/process table entry | C. The pipe buffer | D. A C FILE*

Answer: B. The OS maintains PCB information, and the scheduler decides which runnable process executes.

Solving method: Classify whether the question is about OS process management or C variables.

Slide support: L1 27-30

02 | Lecture 2: More on Processes - MCQ patterns

API	Key exam clue
execl / execlp	List arguments separately; end with (char*)NULL.
execv / execvp	Pass argv vector ending in NULL.
-p suffix	Search PATH; without p, use supplied path.
open / read / lseek	Low-level FD I/O; track returned count and file offset.

1. Successful exec

A child successfully calls execlp("date", "date", NULL). What happens to the following printf() in the old program?

A. Always runs | B. Runs twice | C. Never runs | D. Runs in parent automatically

Answer: C. Successful exec replaces the current program image; the old next statement is never executed.

Solving method: Ask: did exec succeed? A successful exec does not return.

Slide support: L2 10-25, 27-34

---

2. PATH search

Which two calls search the PATH variable to find an executable by name?

A. execl, execv | B. execlp, execvp | C. execl, execlp | D. execv, execvp

Answer: B. The p-suffixed variants search PATH; the others use the supplied path.

Solving method: p = PATH; l = list; v = vector (argv array).

Slide support: L2 22-26

---

3. Variable arguments

You have char *args[] = {"wc", "-l", "file.txt", NULL}. Which call uses this array and searches PATH?

A. execl("wc", args) | B. execvp("wc", args) | C. execlp(args) | D. execv("-l", args)

Answer: B. execvp is the PATH-search + vector variant.

Solving method: Match the argument representation first (array => v), then PATH behavior (=> p).

Slide support: L2 23-26

---

4. Exec failure path

If execlp() returns -1 in a child and the child branch has no exit, what is the risk?

A. Child disappears immediately | B. Child may execute subsequent parent-intended code | C. Parent will always stop | D. Kernel automatically retries

Answer: B. On failure exec returns; the child continues unless explicitly terminated.

Solving method: Look for perror() followed by _exit() after exec.

Slide support: L2 27-34

---

5. FILE* versus FD

Which C I/O family returns a FILE* with stdio buffering?

A. fopen/fread | B. open/read | C. pipe/write | D. dup/close

Answer: A. The standard C stream family uses FILE*, whereas POSIX open returns integer FD.

Solving method: Identify buffered stdio names beginning with f, versus raw FD APIs.

Slide support: L2 35-40

---

6. Open flags

Which open flags create a file if missing and clear it if it exists?

A. O_RDONLY | B. O_WRONLY | O_APPEND | C. O_WRONLY | O_CREAT | O_TRUNC | D. O_RDWR alone

Answer: C. O_CREAT creates and O_TRUNC clears an existing file opened for writing; provide mode e.g. 0644.

Solving method: Translate each flag literally; look for whether creation mode is supplied.

Slide support: L2 37-39

---

7. Offsets via lseek

A file offset is 3. After lseek(fd, 2, SEEK_CUR), where is the offset?

A. 2 | B. 3 | C. 5 | D. End of file

Answer: C. SEEK_CUR adds the provided amount to the current offset.

Solving method: Write old offset + relative movement, not the bytes themselves.

Slide support: L2 35-36

---

8. Inherited descriptors

Without FD_CLOEXEC, what normally happens to existing file descriptors after a successful exec()?

A. All close | B. They remain open | C. Become strings | D. Return to initial 0,1,2 only

Answer: B. Open descriptors are generally inherited across exec unless marked close-on-exec.

Solving method: Distinguish replacing the program from closing file descriptors.

Slide support: L2 40-43

03 | Lecture 3: Redirection - MCQ patterns

Expression	Effect
dup(fd)	New lowest available FD; shared open-file description.
dup2(fd,1)	FD 1 now refers to same resource as fd; normal output redirected.
dup2(fd,0)	FD 0 now refers to same resource as fd; input redirected.
dup2(fd,2)	FD 2 now refers to same resource as fd; error stream redirected.

1. Standard FD numbers

Which FD is standard error?

A. 0 | B. 1 | C. 2 | D. 3

Answer: C. 0 = stdin; 1 = stdout; 2 = stderr.

Solving method: Map the three standard descriptors before tracing changes.

Slide support: L3 2, 10-12

---

2. dup chooses lowest free

Initially FD 0,1,2 are open. fd = open(...) returns 3. What does dup(fd) return if no other FDs opened?

A. 0 | B. 1 | C. 3 | D. 4

Answer: D. dup allocates the lowest currently unused number; 4 is next free.

Solving method: Draw a numbered FD table; cross off closed slots when reusing numbers.

Slide support: L3 12-19

---

3. dup2 destination

After dup2(fd, STDOUT_FILENO), where does a write to descriptor 1 go?

A. Always terminal | B. Same underlying open resource as fd | C. To FD 0 | D. To nowhere

Answer: B. dup2 makes FD 1 refer to the same open file description as fd.

Solving method: Read dup2(oldfd,newfd) as "replace newfd with oldfd's destination."

Slide support: L3 20-34

---

4. Close the original

After dup2(fd,1), close(fd) is called. If FD 1 is still open, can it still write to the redirected file?

A. Yes | B. No | C. Only if fd was 0 | D. Only in the child

Answer: A. Closing the original descriptor does not invalidate the duplicate FD 1.

Solving method: Treat descriptor numbers as separate references to a shared open file description.

Slide support: L3 21, 32-34

---

5. Shared offsets

Two descriptors created with dup(fd) both refer to one regular file. How do sequential writes behave?

A. Always overwrite at offset 0 | B. Share the file offset | C. Open different files | D. Write only one at a time by syntax

Answer: B. Duplicated FDs share the same open file description and its current offset.

Solving method: Track the offset of the open file description, not a separate offset per duplicate.

Slide support: L3 12-19

---

6. Redirect when?

To run wc in a child and redirect only that child's stdout, when should child call dup2()?

A. After successful exec | B. Before fork in parent only | C. In child after fork and before exec | D. Never

Answer: C. The child prepares FD 1 before exec, so the new program inherits it.

Solving method: Put open -> dup2 -> close -> exec inside the child branch.

Slide support: L3 42-55

---

7. Shell redirection syntax

Which shell operator appends standard output to a file?

A. > | B. >> | C. < | D. 2>

Answer: B. >> appends stdout; > truncates/overwrites stdout; 2> redirects stderr; < redirects stdin.

Solving method: Match the stream and whether overwrite or append is requested.

Slide support: L3 6, 9-10

---

8. Redirect before fork

Parent redirects FD 1 to log.txt using dup2, then forks. What does the child inherit?

A. Original stdout only | B. Redirected FD 1 | C. No FDs | D. A second independent log offset

Answer: B. A child inherits the parent's current FD table as of fork().

Solving method: Write a timeline: redirect -> fork versus fork -> redirect.

Slide support: L3 36-41

04 | Lecture 4: Pipes - MCQ patterns

Expression	Effect
p[0]	Read end of anonymous pipe.
p[1]	Write end of anonymous pipe.
read() == 0	EOF only after data exhausted and all write-end references closed.
write() <= PIPE_BUF	No interleaving among writers on POSIX pipe; not a message boundary.

1. Pipe direction

For int p[2]; pipe(p); which endpoint is read-only in the standard one-way interpretation?

A. p[0] | B. p[1] | C. Both identical | D. Neither

Answer: A. p[0] is read end; p[1] is write end.

Solving method: Memorize [0] read / [1] write before every other pipe question.

Slide support: L4 10-20

---

2. Two-way communication

For independent request and response streams between parent and child, what is a simple design?

A. No pipes | B. Two pipes with opposite directions | C. One regular file per byte | D. One child only and no communication

Answer: B. Ordinary anonymous pipes are one-directional; two opposite streams call for two pipes.

Solving method: Draw one arrow in each direction; dedicate one pipe to each.

Slide support: L4 20-22

---

3. Pipe inherited across fork

A parent calls pipe(p) before fork. What does its child inherit?

A. No pipe ends | B. Only p[0] | C. Copies of both FDs | D. A new unrelated pipe

Answer: C. The child inherits descriptors referencing the existing pipe ends.

Solving method: Resources created before fork are accessible to both processes until closed.

Slide support: L4 11-19, 23-25

---

4. EOF conditions

A reader consumed all data. What condition allows read() to return 0 (EOF) on a pipe?

A. Just the pipe is empty | B. Every write-end reference has been closed | C. Every read end is open | D. The parent calls waitpid()

Answer: B. Empty pipe with an open writer can block; EOF follows after all writers close and data is consumed.

Solving method: List all copies of the write end, including dup/fork copies.

Slide support: L4 20-35, 56-58

---

5. Pipe deadlock

Child reads until EOF. Parent writes data, leaves duplicate writer fd open, then waits. Why may both block?

A. Child lost its PID | B. Duplicate writer prevents EOF | C. Integer size is wrong | D. fork() is recursive

Answer: B. The child waits for EOF while the parent waits for the child; an open duplicate write end blocks EOF.

Solving method: Find the cycle "reader waits for EOF; writer owner waits for reader."

Slide support: L4 20-35, 56-58

---

6. read returns

A pipe contains 5 bytes; write ends are then closed. Two calls read(...,3) return what?

A. 3 then 3 | B. 5 then 0 | C. 3 then 2 | D. 0 then 0

Answer: C. The first read consumes 3 bytes; second reads the 2 remaining bytes.

Solving method: Track bytes remaining, not buffer request size.

Slide support: L4 56

---

7. Atomic pipe writes

A single blocking write of <= PIPE_BUF bytes is atomic against other writers. What is guaranteed?

A. Readers always get a full message | B. This write's bytes do not interleave with other writes | C. Writes happen in PID order | D. No writes can block

Answer: B. Atomicity refers to noninterleaving of that write with other writers, not read boundaries or scheduling.

Solving method: Separate atomic write property from read completeness and order.

Slide support: L4 56

---

8. Pipeline redirection

For A | B, what is the correct core setup?

A. A stdin <- write end, B stdout -> read end | B. A stdout -> write end, B stdin <- read end | C. Both children write to read end | D. Parent must execute B with no fork

Answer: B. A produces output into the pipe; B consumes data from the pipe.

Solving method: Draw A -> pipe -> B and translate arrows into dup2() targets 1 and 0.

Slide support: L4 21-35, 36-58

05 | Actual practice-exam MCQ audit (Q1-Q10)

Use this table to check the particular concepts the instructor tested. We summarize the original prompts; this is not a replacement for reading the practice exam.

Question	Answer / most defensible resolution	Method
1 - Parent dies while child runs	B: orphan continues running	Orphan vs zombie distinction.
2 - Print after execvp	B: successful exec does not execute old next statement	Check whether exec succeeds; on failure it returns.
3 - Two forks, grandchild	D: a==0, b==0	First child itself executes second fork, whose child sees b==0.
4 - execl("ls",...)	Technical caveat: supplied path "ls" is not searched through PATH. Under normal directory contents, exec fails (D). A question intending execlp would yield -l listing (B).	Do not silently treat execl as execlp; check whether "ls" exists at the supplied relative path.
5 - fopen vs open	B: FILE* buffered stream vs integer FD low-level I/O	Distinguish standard C stdio and POSIX FDs.
6 - read 10 bytes from pizza	B if file contains "pizza\n" (6 bytes). If exactly "pizza" with no newline, 5; question relies on newline.	Count actual bytes including newline; capacity is not returned length.
7 - return 257	D: normal exit; WEXITSTATUS=1	On typical Unix, 257 modulo 256 = 1.
8 - Pipe endpoints	A and C: p[0] read, p[1] write	Memorize positions before solving.
9 - Pipe statements	A and C: anonymous pipe one-way, can connect multiple processes	A pipe does not automatically broadcast; C is about multiple participants.
10 - dup2 after/before fork	B: child forked after redirection inherits it	Parent change after fork does not retroactively update existing child.

Important: Practice-exam Q4 has a technical ambiguity: execl("ls",...) does not search PATH. Q6 requires knowing whether the one-line file has a trailing newline. These notes preserve the printed wording and explain the assumption rather than presenting an overconfident answer.

Practice-exam short-FRQ structures

Question	Answer shape
Q11 wait vs waitpid	Different target selection + caller blocks + child terminates/reaped + cannot wait on parent.
Q12 four exec variants	Make correct NULL-terminated argv and give execl, execlp, execv, execvp examples.
Q13 descriptors at exit	Assume only 0,1,2 initially open; trace open -> dup -> dup2 -> close, report state just before return. Note missing mode for O_CREAT.
Q14 read through comma	Open file -> read one byte -> break at comma -> store until buffer limit -> append \0 -> close.

06 | Short-answer FRQs: solution methods + worked examples

These 16 questions are new lecture-derived practice, except where a problem is explicitly labeled "practice-exam analogue." They are designed for approximately 5-10 points each.

FRQ 01 | Process count with branching

Question: Starting with one process, execute fork(); if (fork() == 0) _exit(0); How many new children are created in total? Assume successful forks.

```
fork();
if (fork() == 0) _exit(0);
```

How to solve: Step 1: First fork creates 1 new child -> 2 active paths. Step 2: Both paths perform the second fork, creating 2 more children; those newly created children exit.

Answer: Three new children were created, giving four processes in total across the lifetime of the program (including the original). The immediate exits do not erase the fact that the children were created.

Slide support: L1 31-35, 40-50

FRQ 02 | Determine output with independent memory

Question: An original process initializes n=4. Child after fork does n += 6 and prints it; parent waits and prints its n. What outputs?

```
int n=4; pid_t p=fork();
if (p==0) { n+=6; printf("C:%d\n",n); _exit(0); }
waitpid(p,NULL,0); printf("P:%d\n",n);
```

How to solve: Step 1: Copy n=4 into each process. Step 2: Child n=10. Step 3: Parent n stays 4. Step 4: wait makes child output occur first.

Answer: Child prints 10; parent prints 4. Parent and child do not share regular variable modifications.

Careful: Assume child stdout is unbuffered or flushes before _exit, as is typical in output-tracing exercises.

Slide support: L1 24-25, 35-37

FRQ 03 | Compare wait() and waitpid()

Question: Give the difference, what happens to the caller during a blocking wait, and whether either can wait for the caller's parent.

How to solve: Name target selection first, then blocking behavior, then reaping.

Answer: wait() waits for an eligible child; waitpid(pid,&status,0) targets a specific child. A blocking caller suspends until the child changes state (usually terminates). The parent obtains the result and reaps the child. Neither waits on the caller's parent.

Slide support: L1 36-38; practice exam Q11

FRQ 04 | Exec family of four

Question: Write all four ways to run wc -l data.txt; assume /usr/bin/wc exists and wc is on PATH.

```
char *args[]={"wc","-l","data.txt",NULL};
execl("/usr/bin/wc","wc","-l","data.txt",(char*)NULL);
execlp("wc","wc","-l","data.txt",(char*)NULL);
execv("/usr/bin/wc",args);
execvp("wc",args);
```

How to solve: First classify list/vector; then PATH/not; finish every list/array with NULL.

Answer: execl and execv supply /usr/bin/wc; execlp and execvp use wc and PATH. A successful call replaces the running program.

Careful: Four alternative solutions, not four consecutive calls.

Slide support: L2 22-26; practice exam Q12

FRQ 05 | Diagnose exec fallthrough

Question: Find the problem with a child that calls execlp("date", "date", NULL) and then continues into parent-only tasks. Supply a minimal correction.

```
if (pid==0) {
    execlp("date","date",(char*)NULL);
    perror("execlp");
    _exit(127);
}
```

How to solve: Ask whether exec succeeded; if it fails, what code executes next?

Answer: If exec succeeds the old code stops; if it fails it returns -1, and child falls through. Handle failure using perror and _exit(127) inside the child, then keep parent-only tasks in the parent branch.

Slide support: L2 27-34

FRQ 06 | argv and variable arguments

Question: A program is called ./launch 12 7 3. Explain argc, argv[0..3], and which exec variant conveniently forwards an argument array.

How to solve: Count the executable as argv[0]; count user arguments; ensure NULL at argv[argc].

Answer: argc=4. argv[0]="./launch", argv[1]="12", argv[2]="7", argv[3]="3", argv[4]=NULL. execv or execvp accepts an argument array; execvp searches PATH.

Slide support: L2 6-9, 23-26

FRQ 07 | Open flags and the mode argument

Question: Give the flags and mode to create/rewrite report.txt, then describe how to append instead.

```
int fd = open("report.txt",
    O_WRONLY|O_CREAT|O_TRUNC,0644);
if(fd==-1) perror("open");
```

How to solve: Choose O_WRONLY, then add O_CREAT and O_TRUNC; provide mode with O_CREAT. For append, exchange O_TRUNC for O_APPEND.

Answer: open("report.txt",O_WRONLY|O_CREAT|O_TRUNC,0644). To append: open("report.txt",O_WRONLY|O_CREAT|O_APPEND,0644). Both return FD or -1 on failure.

Slide support: L2 35-39; L3 6, 9-10

FRQ 08 | Track lseek and read counts

Question: File holds "abcdefghi". A read of 2 bytes is followed by lseek(fd,3,SEEK_CUR), then read of 2. What are both strings and final offset? Assume buffers are null-terminated.

```
read(fd,a,2);
lseek(fd,3,SEEK_CUR);
read(fd,b,2);
```

How to solve: Start offset 0. First read gets ab -> offset 2. SEEK_CUR +3 -> offset 5. Second read gets fg -> offset 7.

Answer: First text "ab"; second text "fg"; final offset 7. The seek changes position without consuming bytes.

Slide support: L2 35-36

FRQ 09 | Track dup, dup2, and open FDs

Question: Assume only 0,1,2 initially open. fd=open(...) gives 3, copy=dup(fd) gives 4, dup2(copy,1), close(fd). Which FDs remain open and where does FD1 point?

```
int fd=open("out.txt",O_WRONLY|O_CREAT|O_TRUNC,0644);
int copy=dup(fd);
dup2(copy,1);
close(fd);
```

How to solve: Draw numbered columns. open->3; dup->4; dup2 changes 1; close removes 3.

Answer: Immediately afterward FDs 0,1,2,4 remain open. FD1 and FD4 refer to the same open file description as the opened file. On process exit, the OS closes remaining FDs.

Slide support: L3 12-34; practice exam Q13

FRQ 10 | Write stdout and stderr into separate files

Question: Show the essential redirection statements for output FD out and error FD err; explain when to close originals.

```
dup2(out,STDOUT_FILENO);
dup2(err,STDERR_FILENO);
close(out); close(err);
```

How to solve: Open both files; dup2(out,1); dup2(err,2); close originals; use write or printf/fprintf.

Answer: Normal output goes to FD1 file and diagnostics to FD2 file. Closing original out/err does not undo FD1/FD2 redirections.

Slide support: L3 6-13, 20-34

FRQ 11 | Redirect before exec

Question: Explain the order to run wc -l file.txt in a child and send stdout into count.txt. Which process needs to perform dup2?

```
if (pid==0) {
 int out=open("count.txt",O_WRONLY|O_CREAT|O_TRUNC,0644);
 dup2(out,STDOUT_FILENO); close(out);
 execlp("wc","wc","-l","file.txt",(char*)NULL);
 perror("execlp"); _exit(127);
}
```

How to solve: Fork child -> open output in child -> dup2(out,1) -> close(out) -> execlp(wc) -> perror/_exit on error.

Answer: Child performs redirection before exec. Its new wc program inherits stdout pointing at count.txt; parent retains its own stdout.

Slide support: L3 42-55

FRQ 12 | Read bytes up to a delimiter

Question: Using open/read, store bytes before the first comma into a 32-byte char array without overflow. Show the stopping and null-termination logic.

```
char b[32],c; int i=0; ssize_t n;
while((n=read(fd,&c,1))==1){
 if(c==',') break;
 if(i<31) b[i++]=c;
}
if(n==-1) perror("read");
b[i]='\0';
```

How to solve: Open -> read one char while return==1 -> test comma -> store if i<31 -> terminate buffer -> close.

Answer: The comma must not be stored; reserve one byte for \0. Error-check open/read.

Slide support: L2 35-38; practice exam Q14

FRQ 13 | One-way integer pipe

Question: Child computes 2*9 and sends the integer to parent. Identify pipe directions, which ends each process closes, and data size.

```
// CHILD: close(p[0]); int v=18;
write(p[1],&v,sizeof(v)); close(p[1]);
// PARENT: close(p[1]); int got;
read(p[0],&got,sizeof(got)); close(p[0]);
```

How to solve: Create pipe before fork. Child closes read end, writes int, closes write. Parent closes write, reads int, closes read, waits.

Answer: Child uses write(p[1],&value,sizeof(value)); Parent uses read(p[0],&value,sizeof(value)), yielding 18. A complete-transfer check is needed for robust code.

Slide support: L4 10-20

FRQ 14 | Diagnose a pipe deadlock

Question: The child reads until EOF; parent writes data, then calls waitpid before closing its pipe writer. Why can it hang?

How to solve: Write down the dependency cycle. A pipe reader sees EOF only after all writers are closed and data exhausted.

Answer: Child waits for EOF; parent waits for child, but parent still holds write end open. Close all write-end copies before waitpid; then child can observe EOF and finish.

Slide support: L4 20-35, 56-58

FRQ 15 | Explain atomic write vs short read

Question: Two children each write one structure smaller than PIPE_BUF to one pipe. Does the parent always receive one complete structure with each read(sizeof struct)?

How to solve: Separate the atomicity of individual writes from the byte-stream semantics of read.

Answer: Writes <=PIPE_BUF are not interleaved with competing writers, but the parent must still be ready for short reads. Use a loop until sizeof(struct) bytes are collected. Arrival order need not match child number.

Slide support: L4 56, 59

FRQ 16 | Pipeline wiring in short answer

Question: For cat file.txt | tr a-z A-Z, identify which child calls each dup2, which descriptors parent should close, and when exec occurs.

```
// Child A:
dup2(p[1],STDOUT_FILENO);
// Child B:
dup2(p[0],STDIN_FILENO);
// Parent: close(p[0]); close(p[1]);
```

How to solve: One pipe: first child stdout->write end, second child stdin<-read end; close inherited unused ends; both call exec afterward; parent closes both pipe FDs and waits.

Answer: Child A: dup2(p[1],STDOUT_FILENO). Child B: dup2(p[0],STDIN_FILENO). Each closes original pipe ends; exec commands; parent closes both and waits.

Slide support: L4 21-35, 36-58

07 | Mixed self-test: try before revealing answers

One possible practice format: 10 MCQs (3 points each) plus four short answers (5 points each). These are new questions that mirror the practice exam's short-section structure.

M1. The grandchild of the original parent must have what fork return values after `a=fork(); b=fork();`?

A a>0,b>0; B a=0,b>0; C a=0,b=0; D a>0,b=0

M2. Three sequential unconditional forks produce how many total processes?

A 3; B 4; C 6; D 8

M3. Which exec call searches PATH and accepts an argv array?

A execl; B execv; C execvp; D execlp

M4. If open("x",O_RDONLY) succeeds, what does it return?

A FILE*; B nonnegative FD; C PID; D Boolean

M5. An 8-byte buffer is requested by read(fd,b,8), but only 3 bytes remain in an ordinary file. The return value is:

A 8; B 0; C 3; D -1

M6. After dup2(fd,2), an error message written to FD 2 goes:

A stdout; B wherever fd refers; C FD 0; D nowhere

M7. A parent redirects FD1 before forking. The new child receives:

A original FD1; B redirected FD1; C no FDs; D independent stderr

M8. A pipe reader blocks waiting for EOF. Which can explain why?

A missing null terminator; B an unused write FD remains open; C the OS killed the child; D buffer exactly 8 bytes

M9. Which is always correct for ordinary anonymous pipe p?

A p[0] write, p[1] read; B p[0] read, p[1] write; C both broadcast; D no descriptors inherited

M10. A single write of 20 bytes (<PIPE_BUF) is atomic. Which does NOT follow?

A No interleaving with another write; B reader always gets all 20 bytes in one read; C data order within this write preserved; D bytes enter pipe as an uninterrupted sequence

Short-answer self-test

S1. Explain orphan versus zombie and how waitpid relates to zombies.

S2. Give execlp and execvp forms of ls -l with proper NULL termination.

S3. Trace FDs: 0/1/2 open; fd=open(...) returns 3; d=dup(fd); dup2(d,1); close(fd). Which remain open?

S4. Explain why cat | tr needs a pipe, where to dup2 each end, and what the parent closes.

08 | Mixed self-test answer key

Question	Answer	Reason
M1	C	The first child sees a=0 and its child from the second fork sees b=0.
M2	D	2^3 = 8 paths when all processes reach all three forks.
M3	C	v means argument vector and p means search PATH.
M4	B	open returns a nonnegative integer FD on success, -1 on error.
M5	C	read can return fewer bytes than requested; only 3 available.
M6	B	FD 2 becomes a duplicate reference to fd's destination.
M7	B	Fork inherits parent's current FD references.
M8	B	Open writer prevents EOF even if no data remains.
M9	B	p[0] read end; p[1] write end.
M10	B	Pipe atomicity prevents interleaving, not short reads.

Short answers: model responses

S1: An orphan is a running child after its original parent exits; it is reparented. A zombie is a terminated child whose exit status has not been reaped. The parent can use waitpid(childpid, &status, 0) to collect the status and reap it.

S2: execlp("ls", "ls", "-l", (char*)NULL); or char *args[]={"ls","-l",NULL}; execvp("ls",args); Both search PATH and replace the program on success.

S3: fd=3 and d=4. dup2(d,1) redirects FD1, close(fd) removes FD3. Immediately afterward 0,1,2,4 are open. FD1 and FD4 point to the same open file description.

S4: Pipe transfers bytes from cat's stdout to tr's stdin. Child A uses dup2(p[1],1); Child B uses dup2(p[0],0); both close extra ends and exec commands. Parent closes p[0] and p[1] and waits for children.

09 | One-page memory reference

Topic	Memorize / diagnostic question
Fork	Child 0 | Parent positive child PID | failure -1; after fork memory separate.
Order	Without wait/synchronization, print order is not predictable.
Life	Zombie = dead awaiting reap; orphan = parent died before child.
Exec	l=list, v=vector, p=PATH; successful exec never returns.
Open	open -> integer FD; O_CREAT needs mode; read returns actual bytes.
FDs	0 stdin, 1 stdout, 2 stderr; dup lowest free; dup2 chosen target.
Redirection	open -> dup2 -> close original -> exec (child only when child-specific).
Pipe	p[0] read, p[1] write; create before fork; close unused ends.
EOF	Empty pipe is not EOF while any write end remains open.
Atomicity	write <=PIPE_BUF noninterleaving; reads may still be partial.
Exam caveats	execl without p does not PATH-search; "pizza" is 5 bytes, 6 with newline.

Scope and provenance: This guide is an independent study companion synthesized from the four supplied CSE 3100 Part 2 lecture decks and Spring 2026 practice exam. Slide numbers are provided with question patterns; new exercises are clearly separate from the instructor's actual questions. Some technical caveats are identified where a practice-exam item depends on unstated runtime details.